COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	October 14, 2026	**DATE SUBMITTED:**	October 16, 2026
NAME:	Mendoza, Allyson Mendoza, Nathaniel	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 **Algorithmic Strategies**

1. PROCEDURE OUTPUT

Strategy	Algorithm	Analysis
Recursion	Merge Sort	Its process is to continuously split the array until there's 2 elements left to compare and then merge the sorted elements back together. So this means that the work is being done on recursive calls.
Brute Force	Bubble Sort	Because, this sorting algorithm is programmed to check 2 elements at a time. So this keeps comparing until the end. Therefore, it is programmed to exhaustively check.
Backtracking	DFS (Depth First Search)	The program explores the path to its end before it backtracks / returns to the recent node. So it basically checks a specific node till its end or depth before it goes back and checks another node.
Greedy	Selection Sort	Since the algorithm is programmed to scan all unsorted elements and then pick the best one. This is a greedy strategy as it follows through each step without looking at what's ahead or behind.
Divide-and-Conquer	Quick Sort	Because, this is built entirely for divide-and-conquer. Wherein this picks an element as pivot and partitions that array around the picked pivot by placing the pivot in its correct position in the sorted array.

Table 12-1. Algorithmic Strategies and Examples

Screenshot

```
// Memoization
int getMinStepsMemo(int n) {
    if (n == 1) return 0;
    if (memo[n] != -1) return memo[n];

    int r = 1 + getMinStepsMemo(n - 1);
    if (n % 2 == 0) r = min(r, 1 + getMinStepsMemo(n / 2));
    if (n % 3 == 0) r = min(r, 1 + getMinStepsMemo(n / 3));

    memo[n] = r;
    return r;
}
```

```
memset(memo, -1, sizeof(memo));
cout << "Minimum steps to reach 1 (Memoization): " << getMinStepsMemo(n) << endl;
```

```
Enter a number (100 Maximum): 1
Minimum steps to reach 1 (Memoization): 0
```

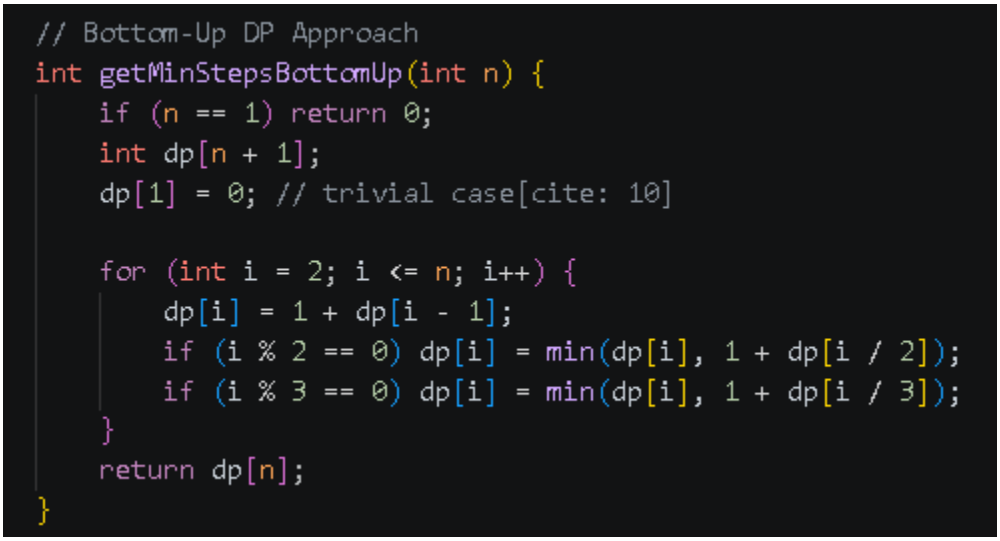
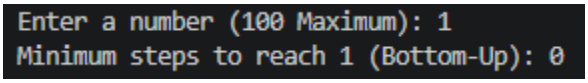
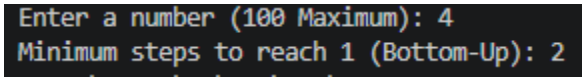
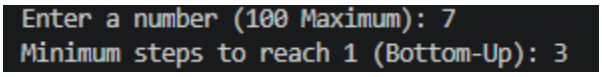
```
Enter a number (100 Maximum): 4
Minimum steps to reach 1 (Memoization): 2
```

```
Enter a number (100 Maximum): 7
Minimum steps to reach 1 (Memoization): 3
```

Analysis

Since the first input is '1', the output is 0 steps because it is already there and so no further steps are needed. With regard to the second input, which is '4', the condition causes it to be divided by 2, that is 4/2 which gives 2, and then it is divided once more by 2 resulting in 1 from 2/2; this therefore amounts to a total of 2 steps. Now concerning the last case with an input of 7: since this is an odd number it is reduced by that amount so it becomes 6, this being counted as one step; then division by 3 is applied so that 6 / 3 is 2, this being another step; and finally it is divided by 2 again to give 1 once more, which is the end of the process, so the total number of steps is 3.

Table 12-2. Memoization Implementation

Screenshot	    
Analysis	The first input is “1” so the output is 0 steps as it is already there so it doesn’t require any further steps. Now For the second input which is “4” it will apply the condition where it will divide itself to 2 which basically becomes 4/2 which results 2 and then divides it again by 2 which results as 1 coming from 2/2 which therefore results 2 steps in total. Now for the last one which has the input of 7. Where this is an odd number it will subtract that number to so it becomes 6 so this counts as 1 step, and then applies division by 3 so (6 / 3) is 2 so it thats another counted step and lastly dividing itself to 2 which results as 1 again which is the end again so it results 3 steps in total.

```
// Bottom-Up DP Approach
int getMinStepsBottomUp(int n) {
    if (n == 1) return 0;
    int dp[n + 1];
    dp[1] = 0; // trivial case[cite: 10]

    for (int i = 2; i <= n; i++) {
        dp[i] = 1 + dp[i - 1];
        if (i % 2 == 0) dp[i] = min(dp[i], 1 + dp[i / 2]);
        if (i % 3 == 0) dp[i] = min(dp[i], 1 + dp[i / 3]);
    }
    return dp[n];
}
```

```
cout << "Minimum steps to reach 1 (Bottom-Up): " << getMinStepsBottomUp(n) << endl;
```

```
Enter a number (100 Maximum): 1
Minimum steps to reach 1 (Bottom-Up): 0
```

```
Enter a number (100 Maximum): 4
Minimum steps to reach 1 (Bottom-Up): 2
```

```
Enter a number (100 Maximum): 7
Minimum steps to reach 1 (Bottom-Up): 3
```

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PSEUDOCODE
FUNCTION countPaths(M, i, j, cost):   IF i < 0 or j < 0   RETURN 0   END IF // out of bounds   IF cost < 0

```
    RETURN 0
  END IF                    // overshot (costs are non-negative)
  IF i == 0 and j == 0
    IF cost == M[0][0]
      RETURN 1
    ELSE
      RETURN 0       // reached start with exact cost
    END IF
  END IF
  RETURN countPaths(M, i-1, j, cost - M[i][j])        // came from above
     + countPaths(M, i, j-1, cost - M[i][j])          // came from left

FUNCTION countMemo(M, i, j, cost, memo):
  IF i < 0 or j < 0
     RETURN 0
  END IF              // out of bounds
  IF cost < 0
    RETURN 0
  END IF             // overshot (costs are non-negative)

  IF i == 0 and j == 0
    IF cost == M[0][0]
      RETURN 1
    ELSE
      RETURN 0       // reached start with exact cost
    END IF
  END IF

  IF memo[i][j][cost] != -1
     RETURN memo[i] [j][cost]
  END IF

  SET memo[i] [j][cost] =
        countMemo(M, i - 1, j, cost - M[i][j], memo) +
        countMemo(M, i, j - 1, cost - M[i][j], memo)
  RETURN memo[i] [j][cost]

MAIN
  SET matrix
  SET cost = 25
  SET M = number of rows in matrix
  SET N = number of columns in matrix

   DISPLAY "Plain recursion: "
   DISPLAY countPaths(mat, M  - 1, N - 1, cost)

  CREATE memo[M][N][cost + 1]
```

ASSIGN all memo values to -1

DISPLAY “Memoized: “
DISPLAY countMemo(mat, M - 1, N - 1, cost, memo)

END MAIN

Table 12-4. Supplementary Pseudocode

CODE

supp.cpp > ...

```cpp
#include <iostream>
#include <vector>
using namespace std;

int countPaths(const vector<vector<int>>& mat, int i, int j, int cost) {
    if (i < 0 || j < 0 || cost < 0) return 0;
    if (i == 0 && j == 0) return (cost == mat[0][0]) ? 1 : 0;
    return countPaths(mat, i - 1, j, cost - mat[i][j]) +
           countPaths(mat, i, j - 1, cost - mat[i][j]);
}

int countMemo(const vector<vector<int>>& mat, int i, int j, int cost,
              vector<vector<vector<int>>>& memo) {
    if (i < 0 || j < 0 || cost < 0) return 0;
    if (i == 0 && j == 0) return (cost == mat[0][0]) ? 1 : 0;
    int& res = memo[i][j][cost];
    if (res != -1) return res;
    return res = countMemo(mat, i - 1, j, cost - mat[i][j], memo) +
                 countMemo(mat, i, j - 1, cost - mat[i][j], memo);
}

int main() {
    vector<vector<int>> mat = {
        {4, 7, 1, 6},
        {5, 7, 3, 9},
        {3, 2, 1, 2},
        {7, 1, 6, 3}
    };
    int cost = 25;
    int M = mat.size(), N = mat[0].size();

    cout << "Plain recursion: " << countPaths(mat, M - 1, N - 1, cost) << endl;

    vector<vector<vector<int>>> memo(M, vector<vector<int>>(N, vector<int>(cost + 1, -1)));
    cout << "Memoized: " << countMemo(mat, M - 1, N - 1, cost, memo) << endl;
    return 0;
}
```

```
Plain recursion: 2
Memoized: 2
```

Table 12-5. Supplementary Code and Output

MANUAL COMPUTATION

FIRST PATH			
4	7	1	6
5	7	3	9
3	2	1	2
7	1	6	3

SECOND PATH			
4	7	1	6
5	7	3	9
3	2	1	2
7	1	6	3

Path Computation	Path Direction
Path 1 : 4 + 5 + 7 + 3 + 1 + 2 + 3 = 25	*Down, Right, Right, Down, Right, Down*
Path 2 : 4 + 7 + 1 + 3 + 1 + 6 + 3 = 25	*Right, Right, Down, Down, Down, Right*

PROOF / JUSTIFICATION (*That there's only two (2) possible paths*)

Path #	Moves	Total	Path #	Moves	Total
1	RRRDDD	32	11	DRRRDD	33
2	RRDRDD	29	12	DRRDRD	**25**
3	RRDDRD	21	13	DRRDDR	29
4	RRDDDR	**25**	14	DRDRRD	24
5	RDRRDD	35	15	DRDRDR	28
6	RDRDRD	27	16	DRDDRR	28
7	RDRDDR	31	17	DDRRRD	20
8	RDDRRD	26	18	DDRRDR	24
9	RDDRDR	30	19	DDRDRR	24
10	RDDDRR	30	20	DDDRRR	29

Analysis (Mendoza, Allyson)
The program produces the same output using the two methods which are the plain recursion and memoization. Which means there are two possible ways in order to get specific value. It starts from the bottom-right cell to the top-left cell of the given matrix wherein the sum of all the visited cells are equal to 25. In Plain Recursion, it can lead to repeated calculations. While in Memoization, it prevents unnecessary repeated calculations. But, both methods are working and they both display 2 which confirms that they produce the same output for the given input.
Analysis (Mendoza, Nathaniel)
The program counts how many right-and-down paths from the top-left to the bottom-right add up to the target cost. It works backwards from the last cell, adding the counts from the cell above and the cell to the left. It returns 0 when a path goes out of bounds or overspends, and 1 when it reaches the start with the exact cost. Memoization stores each (row, column, remaining cost) result. In addition to this approach, in order to check and know if the code produces the expected output, Brute Force manual computation was used for this. Wherein the made table above shows and proves that there's only 2 possible paths for the given matrix.

CONCLUSION
Mendoza, Allyson
In conclusion, this hands-on activity helped me understand how recursion and memoization works. Where in recursion, it works by dividing them into subproblems and it uses base cases, recursive calls, and conditional statements. While in memoization, that can store the results of the previous solved subproblems in order to avoid repeated calculations. Overall, combining recursive and memoization can work efficiently when it comes to solving problems. Also, it helped me understand which algorithm is the best to use and how it can be applied in solving problems.
Mendoza, Nathaniel
After performing this laboratory activity, I was able to recall the strategies that used the following algorithms, specifically the sorting ones. Additionally, I learned how to optimize and modify the code in a shorter way . This activity taught me that the obvious choice is not always the best one, like greedy failing in Minimum Steps to One, and that saving past results makes a program much faster. Lastly, I recalled the usage of matrix and its manual solving.

REFERENCE
Number of paths with exactly k coins - GeeksforGeeks - Supplementary

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.